

UNIVERSIDAD MAYOR DE SAN ANDRES

CARRERA DE INFORMATICA

PROGRAMACION WEB III

INF-133



PRACTICA NRO 1

Estudiante : Ticona Tapia Jhonatan Gerardo

CI:13971678

Docente: Lic. Jhonny Felipez Andrade

Fecha: 22 de septiembre 2025

LA PAZ - BOLIVIA

2025

1. Crear una función que cuente cuántas veces aparece cada vocal en un texto y devuelva el resultado en un objeto.

```
let contador = { a: 0, e: 0, i: 0, o: 0, u: 0 };
function miFuncion(texto) {
  for (let letra of texto.toLowerCase()) {
    if ("aeiou".includes(letra)) {
      contador[letra]++;
    }
  }
  return contador;
}
let obj = miFuncion("euforia");
console.log(obj);
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio1
{ a: 1, e: 1, i: 1, o: 1, u: 1 }
```

2. Crear una función que invierta el orden de las palabras en una frase.

```
1 function miFuncion(cad) {
2   return cad.split("").reverse().join("");
3 }
4
5 let cad = miFuncion("abcd");
6 console.log(cad);
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio2
dcba
```

3. Crear una función que reciba un arreglo de números y devuelva en un objeto a los pares e impares:

```
1 let numeros = { pares: [], impares: [] };
2 function miFuncion(x) {
3   for (let num of x) {
4     if (num % 2 === 0) {
5       numeros.pares.push(num);
6     } else {
7       numeros.impares.push(num);
8     }
9   }
10  return numeros;
11 }
12 let obj = miFuncion([1, 2, 3, 4, 5]);
13 console.log(obj);
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio3
{ pares: [ 2, 4 ], impares: [ 1, 3, 5 ] }
```

4. Crear una función que reciba un arreglo de números y devuelva el número mayor y el menor, en un objeto.

```
function miFuncion(v) {
  return {
    mayor: Math.max(...v),
    menor: Math.min(...v)
  };
}
let obj = miFuncion([3, 1, 5, 4, 2]);
console.log(obj);
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio4
{ mayor: 5, menor: 1 }
```

5. Crear una función que determine si una cadena es palíndromo (se lee igual al derecho y al revés).

```
function miFuncion(cad) {  
  let t = cad.toLowerCase();  
  let n = t.length;  
  
  for (let i = 0; i < n / 2; i++) {  
    if (t[i] !== t[n - 1 - i]) {  
      return false;  
    }  
  }  
  return true;  
}  
  
console.log(miFuncion("oruro"));  
console.log(miFuncion("hola"));
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio5  
true  
false
```

6. Tomar los dos primeros elementos de un arreglo y almacenarlos en dos variables mediante desestructuración.

```
let numeros = [2,5,7,8,4,1];  
let [a, b] = numeros;  
console.log(a);  
console.log(b);
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio6  
2  
5
```

7. Almacenar el resto de los elementos de un arreglo sin tomar en cuenta los dos primeros elementos de un arreglo, mediante desestructuración.

```
let numeros = [1,2,3,4,5,6];  
let [, , ...resto] = numeros;  
console.log(resto);
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio7  
[ 3, 4, 5, 6 ]
```

8. Realizar un código para ejecutar una función callback después 2 segundos.

```
function miFuncion(callback) {  
  setTimeout(callback, 2000);  
}  
  
miFuncion(function() {  
  console.log("este mensaje se mostro despues de 2s");  
});
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio8  
este mensaje se mostro despues de 2s
```

9. Crear una promesa que devuelva un mensaje de éxito después de 3 segundos.

```
function miPromesa() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve("¡Éxito!");  
    }, 3000);  
  });  
}  
  
miPromesa().then(mensaje => {  
  console.log(mensaje);  
});
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio9  
¡Éxito!
```

10. ¿Cuándo es conveniente utilizar un callback, y cuando es necesario utilizar una promesa?

Se usa **callbacks** para tareas simples o cuando trabajes con librerías que no soporten promesas, se usa **promesas** (o `async/await`) para operaciones más complejas, encadenadas o cuando necesites un manejo de errores más robusto y un código más legible.

Ambas herramientas son útiles, pero las promesas (y `async/await`) son más modernas y recomendadas para la mayoría de los casos en proyectos actuales.

11. Proporcione un ejemplo concreto de encadenamiento de promesas.

```
function primeraPromesa() {  
  return new Promise((resolve) => {  
    resolve(console.log("Primera promesa"));  
  });  
}  
function segundaPromesa() {  
  return new Promise((resolve) => {  
    resolve(console.log("Segunda promesa"));  
  });  
}  
function terceraPromesa() {  
  return new Promise((resolve) => {  
    resolve(console.log("Tercera promesa"));  
  });  
}  
primeraPromesa()  
  .then(segundaPromesa())  
  .then(terceraPromesa())  
  ;
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio11  
Primera promesa  
Segunda promesa  
Tercera promesa
```

12. Proporcione un ejemplo concreto donde el anidamiento de callbacks se puede reescribir mejor con `async/await` haciendo el código más limpio y mantenible.

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio12  
Paso 1  
Paso 2  
> Paso 3  
□ Resultado: 25
```

```
function paso1Async() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      console.log("Paso 1");  
      resolve(10);  
    }, 1000);  
  });  
}  
  
function paso2Async(valor) {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      console.log("Paso 2");  
      resolve(valor * 2);  
    }, 1000);  
  });  
}  
  
function paso3Async(valor) {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      console.log("Paso 3");  
      resolve(valor + 5);  
    }, 1000);  
  });  
}  
  
async function ejecutar() {  
  const result1 = await paso1Async();  
  const result2 = await paso2Async(result1);  
  const result3 = await paso3Async(result2);  
  console.log("Resultado:", result3);  
}  
ejecutar();
```

13. Proporcione un ejemplo concreto donde el anidamiento de promesas se puede reescribir mejor con async/await haciendo el código más limpio y mantenible.

```
function primeraPromesa() {  
  return Promise.resolve(console.log("Primera promesa"));  
}  
function segundaPromesa() {  
  return Promise.resolve(console.log("Segunda promesa"));  
}  
function terceraPromesa() {  
  return Promise.resolve(console.log("Tercera promesa"));  
}  
async function ejecutar() {  
  await primeraPromesa();  
  await segundaPromesa();  
  await terceraPromesa();  
  console.log("Todas las promesas completadas");  
}  
ejecutar();
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio13  
Primera promesa  
Segunda promesa  
Tercera promesa  
Todas las promesas completadas
```

14. Proporcione un ejemplo para convertir una promesa en un callback.

```
function miPromesa() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve("Promesa completada");  
    }, 1000);  
  });  
}  
function promesaACallback(callback) {  
  miPromesa()  
    .then(result => callback(null, result))  
    .catch(error => callback(error));  
}  
promesaACallback((err, resultado) => {  
  if (err) {  
    console.error("Ocurrió un error:", err);  
  } else {  
    console.log("Resultado:", resultado);  
  }  
});
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio14  
Resultado: Promesa completada
```

15. Proporcione un ejemplo para convertir un callback en una promesa.

```
function sumarAsync(a, b, callback) {
  setTimeout(() => {
    if (typeof a !== 'number' || typeof b !== 'number') {
      callback("Los parámetros deben ser números");
    } else {
      callback(null, a + b);
    }
  }, 1000);
}

function sumarPromesa(a, b) {
  return new Promise((resolve, reject) => {
    sumarAsync(a, b, (err, result) => {
      if (err) reject(err);
      else resolve(result);
    });
  });
}

sumarPromesa(2, 3)
  .then(result => console.log("Resultado de la promesa:", result))
  .catch(err => console.error(err));
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio15
Resultado de la promesa: 5
```

16. Proporcione un ejemplo para migrar una función con promesas a async/await.

```
function obNumero() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(10);
    }, 1000);
  });
}

function multPorDos(num) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(num * 4);
    }, 1000);
  });
}

async function ejecutar() {
  try {
    const num = await obNumero();
    const resultado = await multPorDos(num);
    console.log("Resultado:", resultado);
  } catch (err) {
    console.error(err);
  }
}

ejecutar();
```

```
E:\jhona\Programacion2\Practica1inf133>node ejercicio16
Resultado: 40
```